Домашнее задание 2

```
Каждое задание оценивается в **2 балла**
```

---

1. Банковские операции

Откройте CSV-файл `transactions.csv` с банковскими транзакциями.
В нём шесть колонок: `transaction_id`, `client_id`, `transaction_sum_roubles`, `purpose`, `card_type`, `transaction_datetime`.
Первая строка - заголовок, файл в кодировке UTF-8.

Задание:

- Прочитайте этот файл штатным модулем `csv`, без `pandas`. (для чтения используйте `csv.DictReader`)
- Прочитать нужно в удобную *простую структуру* (не класс), не забудьте привести данные к нужному типу (csv читает все в `str`).
- Для этой структуры посчитайте:
  - Сумму по полю `transaction_sum_roubles`
  - Сумму по полю `transaction_sum_roubles` в разрезе кредитных и дебитовых карт (debit / credit)
  - Агрегат: среднее значение по полю `transaction_sum_roubles` в разрезе поля `purpose`

Использовать можно только **стандартные библиотеки**, результаты вычислений можете складывать в отдельные переменные, агрегаты в `dict`. Приветствуется использование `lambda` выражений, где это возможно. Не забывайте про аннотации. Используйте только стандартные библиотеки.

2. Кеширование

Дана функция:

```
import time

def _slow_square(number: int) -> int:
    time.sleep(3)
    return number * number
```

Функцию `_slow_square` нельзя изменять.

Задание Создайте функцию `cached_square` которая внутри себя вызывает `slow_square`:

- При вызове первом `cached_square(значение1)` функция выполняется около 3 секунд - происходит вызов `slow_square`
- При последующих вызовах с `cached_square` тем же значением `значение1` функция сразу возвращает результат

Примеры

```
cached_square(2) = 4    Время выполнения ~3сек
cached_square(2) = 4    Время выполнения ~0сек
cached_square(3) = 9    Время выполнения ~3сек
cached_square(2) = 4    Время выполнения ~0сек
cached_square(5) = 25   Время выполнения ~3сек
```

Подсказка Используйте *упорядоченную структуру* рассмотренную в теме 3.

3. Функции 1: позиционные аргументы

Задание Напишите функцию `polynomal_formatter` которая:

- На вход может получить произвольное число *позиционных аргументов*
- Аргументы - коэффициенты от старшей степени к младшей,
- Возвращает строку уравнения, приравненного к нулю.
- Использует только стандартные библиотеки

Примеры

```
polynomial_formatter(3, 2, 1)     -> "3*x^2 + 2*x + 1 = 0"
polynomial_formatter(1, -2, 1)    -> "x^2 - 2*x + 1 = 0"
polynomial_formatter(0, 2, 1)     -> "2*x + 1 = 0"
```

4. Функции 2: именованые аргументы

Задание Напишите функцию `select_top_20`, которая:

- Принимает на вход датасет из задания 1 Используйте *простую структуру* датасета `transactions` из задания 1.
- Возвращает топ 20 значений по **произвольным** входным фильтрам
- Использует только стандартные библиотеки

Примеры

```
select_top_20(transaction_id=9) -> [{'transaction_id': 9,
                                     'client_id': 5093997654,
                                     'transaction_sum_roubles': 142844.2115,
                                     'purpose': 'Electronics',
                                     'card_type': 'debit',
                                     'transaction_datetime': '2026-06-04
```

```
00:50:14'}]
select_top_20(purpose="Fuel", card_type="debit") -> [{'transaction_id': 10,
                                                     'client_id':
1559552960,

'transaction_sum_roubles': 326636.9294,
                                                     'purpose': 'Fuel',
                                                     'card_type':
'debit',

'transaction_datetime': '2026-06-04 01:20:50'},
                                                    {'transaction_id':
12,
                                                     'client_id':
3694885437,

'transaction_sum_roubles': 186316.0524,
                                                     'purpose': 'Fuel',
                                                     'card_type':
'debit',

'transaction_datetime': '2026-06-26 21:02:39'},
                                                    ................]
```

5. Декоратор

Задание Реализуйте декоратор `@log_func_call`, который логирует каждый вызов декорируемой функции.

- Перед вызовом функции декоратор печатает строку вида: `Вызов: имя_функции(арг1, арг2, ключ=значение)`
- После успешного вызова печатает строку вида: `Результат: имя_функции(...) -> значение`
- Если функция выбросила исключение - выводим: `Ошибка: имя_функции(...) -> ТипИсключения: сообщение`
- (*Задание со звездочкой*) Добавьте параметр `@log_func_call(show_args=True, show_result=True)`, чтобы можно было включать/выключать вывод аргументов и результата независимо друг от друга.

Пример использования:

```
@log_func_call
def add(a, b):
    return a + b

@log_func_call
def divide(a, b):
    return a / b

add(2, 3)
```

```
# Вызов: add(2, 3)
# Результат: add(2, 3) -> 5

divide(10, 0)
# Вызов: divide(10, 0)
# Ошибка: divide(10, 0) -> ZeroDivisionError: division by zero
# Traceback ...
```

Используйте только стандартные библиотеки.

6. ООП

Задание Реализуйте классы: `LinearSpaceElement`, `Vector2D` (вектор в двумерном пространстве) свяжите их (подумайте как) В `markdown` ячейке или файле объясните связь между классами.
Класс `Vector2D` имеет всего 2 внутренние переменные `x` и `y`.
Класс `Vector2D` должен поддерживать:

- Сложение вычитание векторов
- Умножение и деление вектора на число
- Скалярное произведение векторов
- Вывод длины вектора (через метод)

Создание вектора:

```
a = Vector2D(3, 4)
b = Vector2D(1, 2)
```

Сложение и вычитание векторов:

```
a + b   # Vector2D(4, 6)
a - b   # Vector2D(2, 2)
```

Используйте только стандартные библиотеки

7. Классы и функции при работе с API

Задание Дана функция `fetch_users`, которая имитирует вызов API.
Функции `fetch_users` и `_make_user` **нельзя менять**.

```
import random
import string

def _make_user(user_id: int) -> dict:
    return {
        "id": user_id,
        "name": "".join(random.choices(string.ascii_uppercase, k=5)),
```

```
        "age": random.randint(18, 60),
        "email": "".join(random.choices(string.ascii_lowercase, k=6)) +
"@gmail.com",
    }


def fetch_users(user_id: int | None = None) -> list[dict]:
    if user_id is None:
        ids = range(1, random.randint(2, 6))
    elif random.random() < 0.2:
        return []
    else:
        ids = [user_id]

    return [_make_user(i) for i in ids]
```

- Реализуйте класс `User`
- Реализуйте функцию `user_factory`, которая принимает на вход API-функцию `fetch_users`
- Используя `user_factory` лямбду, получите:
  - список всех пользователей;
  - пользователя с `id=101`;
  - пользователей у которых `age>20`;

Пример использования:

```
get_users = user_factory(fetch_users)

all_users = get_users()          # [User(...), User(...), ...]
one_user  = get_users(2)         # User(id=2, ...) или []
nobody    = get_users(99)        # User(id=99, ...) или []
```

Используйте только стандартные библиотеки

8. Банковский счёт

Задание Реализуйте два класса: `BankAccount` и своё исключение `InsufficientFundsError`.

Класс `BankAccount`:

- Переменные: `owner`, `_balance` (баланс должен быть приватным)
- Методы: `deposit` - пополнение, `withdraw` - снятие со счёта (при нехватке средств нужно выдать ошибку)
- Переопределены магические методы

Класс исключения:

```
class InsufficientFundsError(Exception):
```

```
    <ваш код тут>
```

Пример

```
acc = BankAccount("Иван")
acc.deposit(1000)
acc += 500
acc -= 200

print(acc)              # Счёт Иван: 1300 руб.
print(acc.balance)      # 1300
print(bool(acc))        # True

try:
    acc.withdraw(5000)
except InsufficientFundsError:
    print("Недостаточно средств")
```

Используйте только стандартные библиотеки.